



**JSS MAHAVIDYAPEETHA**  
**JSS Science and Technology University**  
**Sri Jayachamarajendra College of Engineering**



**DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION  
ENGINEERING**

**DIGITAL SIGNAL PROCESSING  
(22EI540)**

**EVENT II**  
**Audio Restoration for Vinyl Records Using Adaptive Filters**

**Submitted By:**

| <b>Student Name</b>   | <b>USN</b>    |
|-----------------------|---------------|
| 1. SAMRUDH M          | 02JST22UEI043 |
| 2. SUJIT SATISH HEGDE | 02JST22UEI052 |
| 3. VIRUPAKSH A ALUR   | 02JST22UEI059 |

**Submitted To:**  
**Ms. Rajeswari P**  
Assistant Professor  
Department of Electronics and Instrumentation  
JSSSTU, Mysuru-06

## **CONTENTS:**

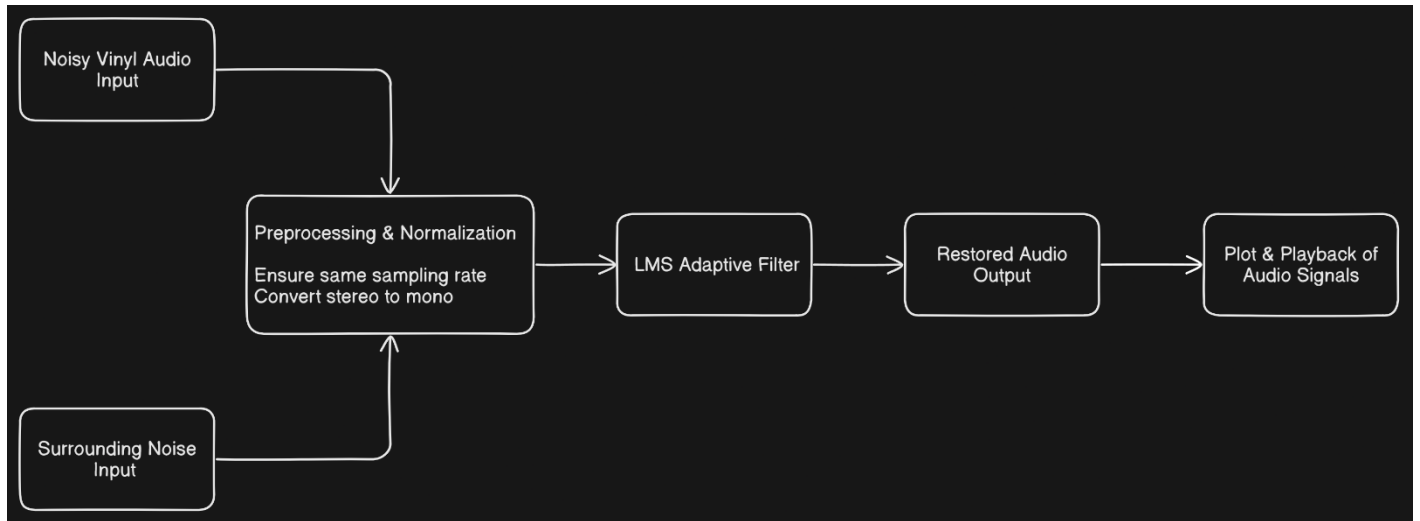
1. INTRODUCTION.
2. BLOCK DIAGRAM.
3. WORKING.
4. PROGRAM.
5. FLOWCHART AND ALGORITHM.
6. OUTPUT.
7. CONCLUSION.

## 1.INTRODUCTION:

Vinyl records are known for their warm and rich sound, but they often suffer from noise artifacts such as pops, crackles, and background hum due to dust, scratches, and analog imperfections. Traditional noise reduction techniques may struggle to preserve the original quality while effectively removing unwanted disturbances. To address this, adaptive filtering techniques, particularly the **Least Mean Squares (LMS) adaptive filter**, are used to dynamically remove noise while preserving the integrity of the audio signal.

The LMS adaptive filter operates by continuously adjusting its filter coefficients to minimize the difference between the noisy vinyl audio and a reference noise signal. This method ensures real-time noise suppression while adapting to varying noise patterns, making it well-suited for vinyl restoration. By utilizing an external noise reference, such as a recording of the ambient noise affecting the vinyl, the LMS algorithm can efficiently extract and cancel out the noise component, resulting in a **cleaner and more authentic audio experience**.

## 2. BLOCK DIAGRAM:



**Figure 1 : Block Diagram**

## 3.WORKING:

The Least Mean Squares (LMS) adaptive filter is used in this project to remove noise from vinyl records by continuously adjusting its filter coefficients to minimize the error between the noisy input and the desired output. The restoration process follows these key steps:

### 1. Input Acquisition:

- The system takes two audio inputs:
  - Noisy Vinyl Audio (containing both the original audio and unwanted noise).
  - Reference Noise Signal (a recording of surrounding noise, such as hum, crackles, or background disturbances).
- Both audio signals are converted to mono and sampled at the same rate to ensure proper processing.

### 2. Adaptive Filtering Using LMS Algorithm:

- The LMS filter is initialized with a small step size ( $\mu$ ) and a filter length (number of taps).

- The filter processes the reference noise signal and adjusts its coefficients dynamically to match the noise component in the vinyl audio.
- The LMS algorithm updates its weights based on the equation:
- $w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n)$ 
  - $w(n) \rightarrow$  Filter coefficients
  - $\mu \rightarrow$  Step size for convergence
  - $e(n) \rightarrow$  Error signal (difference between noisy vinyl audio and estimated noise)
  - $x(n) \rightarrow$  Reference noise input

### 3. Noise Cancellation:

- The LMS filter **estimates the noise** present in the vinyl recording using the reference signal.
- This estimated noise is **subtracted** from the noisy vinyl signal, leaving behind a cleaner version of the original audio.

### 4. Output Processing and Visualization:

- The restored audio signal is saved as a .wav file.
- A time-domain plot is generated to compare the noisy input, reference noise, and the restored output.
- The system also plays both the original noisy audio and the cleaned audio for subjective evaluation.

### 4.PROGRAM:

```

clc;
clear all;
close all;
% Hardcode file paths for testing (use your own paths)
vinylFile = 'E:\hello\noise.wav';
[vinylSignal, Fs] = audioread(vinylFile);
noiseFile = 'E:\hello\orig.wav';
[noiseSignal, Fs_noise] = audioread(noiseFile);
% Ensure both files have the same sampling rate
if Fs ~ Fs_noise

```

```

error('Sampling rates of the vinyl and noise files must be the same!');
end
% Convert to mono if stereo
if size(vinylSignal, 2) > 1
    vinylSignal = mean(vinylSignal, 2);
end
if size(noiseSignal, 2) > 1
    noiseSignal = mean(noiseSignal, 2);
end
% Truncate or pad noise signal to match the length of vinyl signal
minLen = min(length(vinylSignal), length(noiseSignal));
vinylSignal = vinylSignal(1:minLen);
noiseSignal = noiseSignal(1:minLen);
% LMS Adaptive Filter parameters
mu = 0.00001; % Step size
numTaps = 520; % Filter length
adaptiveFilter = dsp.LMSFilter('Length', numTaps, 'StepSize', mu);
% Apply LMS adaptive filtering
[filteredSignal, ~] = adaptiveFilter(noiseSignal, vinylSignal);
% Save the restored audio
outputFile = 'restored_audio.wav';
audiowrite(outputFile, filteredSignal, Fs);
disp(['Restored audio saved as: ', outputFile]);
% Plot original and filtered signals
t = (0:minLen-1) / Fs;
figure;
subplot(3,1,1);
plot(t, vinylSignal);
title('Noisy Vinyl Audio');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3,1,2);
plot(t, noiseSignal);
title('Surrounding Noise');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3,1,3);

```

```

plot(t, filteredSignal);
title('Restored Audio');
xlabel('Time (s)');
ylabel('Amplitude');
% Play original and restored audio
disp('Playing noisy vinyl audio...');
sound(vinylSignal, Fs);
pause(length(vinylSignal)/Fs + 1);
disp('Playing restored audio...');
sound(filteredSignal, Fs);

```

## 5. FLOWCHART AND ALGORITHM:

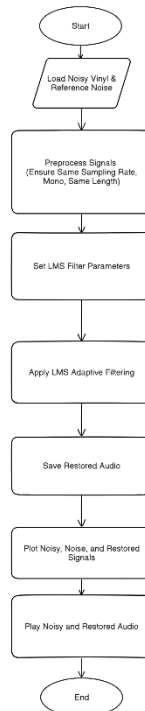


Figure 2: Flowchart

### Input Files:

Load the noisy vinyl audio (vinylSignal) and surrounding noise (noiseSignal) from their respective files.

### Preprocessing:

Ensure both audio files have the same sampling rate.

Convert stereo signals to mono (if necessary).

Truncate the signals to ensure they are the same length.

### **LMS Adaptive Filter Setup:**

Set the step size ( $\mu$ ) and filter length (numTaps).

Initialize the LMS adaptive filter.

### **Noise Removal:**

Apply the LMS adaptive filter to the noise reference signal (noiseSignal) and noisy vinyl audio (vinylSignal).

The filter adjusts its weights to subtract the noise from the vinyl signal, producing the filtered signal (filteredSignal).

### **Save Restored Audio:**

Write the cleaned audio (filteredSignal) to a new .wav file.

Visualization & Playback:

### **Plot the waveforms of:**

Noisy vinyl audio.

Surrounding noise.

Restored audio.

Play the noisy vinyl audio and the restored audio for comparison.



## 6. OUPUT:

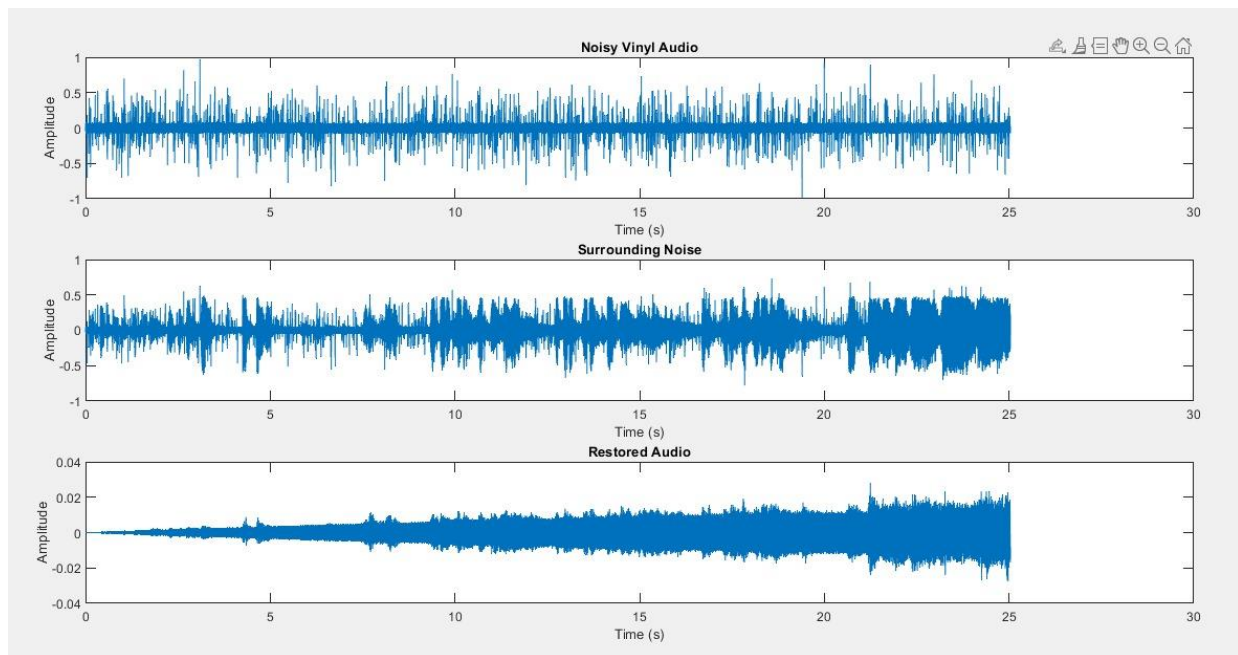


Figure 3

a)Noisy Vinyl Audio

b) Surrounding Noise

c)Restored Audio

## 7. CONCLUSION:

- This project effectively demonstrates how **adaptive filtering** techniques can be applied to **restore noisy vinyl audio**. By using an **LMS adaptive filter**, we were able to significantly reduce background noise, providing a cleaner, high-quality audio output.
- The process not only preserves the integrity of the original sound but also provides a practical solution for audio restoration tasks, particularly for those working with old recordings or vinyl records.
- In conclusion, **adaptive filtering** offers a promising approach to **audio restoration**, with potential for further refinement and application in both academic and industry-level audio processing tasks.

